```
class Example
  include GeneralLogger

  attr_logger :value
end

ex = Example.new
ex.log("New example created")
ex.value = 123
puts "Value is #{ex.value}"
ex.value = "cat"
puts "Value is #{ex.value}"
```

produces:

```
00:08: New example created
00:08: Assigning 123 to value
Value is 123
00:08: Assigning "cat" to value
Value is cat
```

Notice how the included callback is used to extend the host class with the methods defined in the inner module ClassMethods. Ruby on Rails uses this pattern extensively using the name *Concern* for such a module that provides both class and instance functionality when mixed in and providing shortcut methods so you don't need to add the included hook yourself.

Now, as an exercise, try walking through the previous example in your head. For each line of code, work out the value of self. Master this, and you've pretty much mastered this style of metaprogramming in Ruby.

Struct and Subclassing Expressions

This is a generalization of the regular class definition syntax. You know that you can write this:

```
class Parent
  ...
end

class Child < Parent
  ...
end
```

What you might not know is that the thing to the right of the < needn't be just a class name; it can be any expression that returns a class object. In this code example, we have the constant Parent. A constant is a simple form of expression, and in this case the constant Parent holds the class object of the first class we defined.

Ruby comes with a class called Struct, which allows you to define classes that contain just data attributes. For example, you could write this:

```
Person = Struct.new(:name, :address, :likes)

dave = Person.new('Dave', 'TX')
dave.likes = "Programming Languages"
puts dave
```

produces:

```
#<struct Person name="Dave", address="TX", likes="Programming Languages">
```